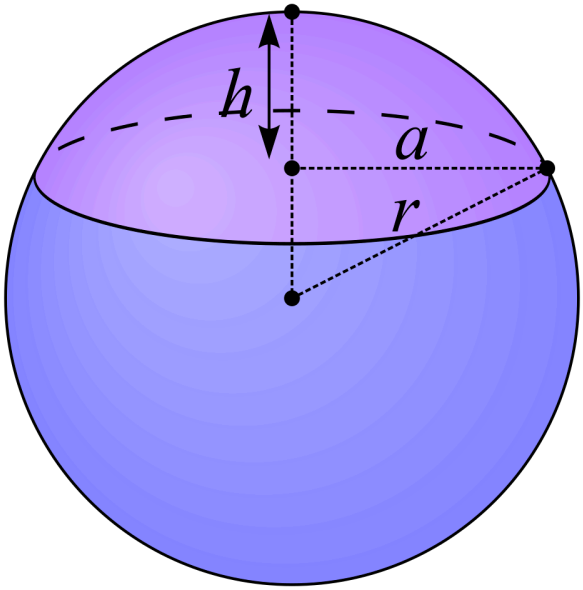
CAPS

Home * Engines * CAPS

CAPS, (Chess as Problem Solving, CAPS-II)
Hans Berliner's experimental chess program which was subject of his 1974 Ph.D. thesis *Chess as Problem Solving: The Development of a Tactics Analyser* at Carnegie Mellon University [2], further eloborated at the Advances in Computer Chess 1 conference [3], and, along with B*, in a Panel on Computer Game Playing [4]. CAPS is a selective, knowledge driven depth-first alpha-beta searcher in the domain of chess tactics. A position is represented as vector of 1040 words of information, including the list of generated pseudo-legal moves, kept on a stack of up to 20 ply.



Spherical Cap [1]

Contents

- 1 CAPS-II
- 2 Refutation Description
- 3 Causality Facility
- 4 Method of Analogies
- 5 Quotes
- 6 See also
- 7 Publications
- 8 Forum Posts
- 9 External Links
  - 9.1 Caps
  - 9.2 Cap
- 10 References

CAPS-II

CAPS-II was tested on many middle-game chess tactics problems from standard textbooks on chess. It played a few complete games of chess. In both these modes it ran with a maximum depth of 10 ply. CAPS-II did not do too well in the games, since it had little positional knowledge, and more importantly the tactics mechanisms were still

not completed, causing it to make occasional blunders which would wipe out whatever good it had achieved earlier. However, it did quite well on the tactics problems such as Reinfeld's (https://en.wikipedia.org/wiki/Fred_Reinfeld) Win at Chess [5].

Refutation Description

While visiting a node, CAPS determines attack information, and whether pieces are completely or partial en prise or overprotected, are pinned, or may be source of a discovered attack, and further collects a so called **Refutation Description** of expandable sets (bitboards and piece-sets) of moving and captured pieces, their source- and target squares, and appropriate paths of sliding pieces involved, associated with the full qualified moves (piece, from, to, captured piece if any) along the current search line. The description of the six sets of the Refutation Description is given from the Advances in Computer Chess 1 paper [6]:

- **RPCS** - is a bit-vector which has bits representing names of pieces. The name bit <code>of the piece that moved to produce this node is set in this vector.
- **RSQS** - is a bit-vector with bits representing squares on the board. The bit corresponding to the destination square of the move that produced this node is set in RSQS.
- **RPATH** - is a bit-vector with bits representing squares on the board. The bit for any square across which a sliding piece moved in making the made move is set in RPATH. If the move was a non-capture pawn move, then all squares over which it passed including the destination also have bits set for them.
- **RTGTS** - is a bit-vector with bits representing names of targets. A comparison is made of BEST for this node with BEST one ply previously. Any squares which are now named as containing material targets, but were not mentioned in the previous BEST, have bits set for the name of the piece on this square to indicate that this threat was created by the last move.
- **TGTSQS** - is a bit-vector with bits representing squares on the board. For any RTGTS detected as above, bits are set in TGTSQS for the corresponding squares.
- **TPATH** - is a bit-vector with bits representing squares on the board. For any TGTSQS detected as above, if a piece that has an ATTACKING function on this square is a sliding piece, then all the intervening squares have bits set for them in TPATH.

Causality Facility

While classical depth-first searchers may perform the killer heuristic, or may analyze the backed-up principal variation for refutations, i.e. to move or defend any captured piece, or capture or pin any capturer, or block the path of any moving piece, CAPS' refutation description allows for much more complete understanding of a set of consequences, not only restricted to the PV. The **Causality Facility** interprets the refutation descriptions and tries to determine tactical causes and goals to control a set of move generators for possible goal transitions, and allows pruning of large sub-trees. The facility may further execute a null move to determine whether a suspected move was responsible for a bad position or not.

Method of Analogies

Once a move was determined as in fact bad, it is possible to posit a Lemma (https://en.wikipedia.org/wiki/Lemma_%28mathematics%29) along with its environment of the refutation description — this being knowledge that a certain move will be bad as long as the described environment does not change. With this knowledge, any proposed move may be looked up in the Lemma file and if it has been previously cataloged, the program may determine if

the current position contains any essential changes from the Lemma environment which might make the move succeed. It is important to note that should it be decided to try the move, and should it again fail, that it would now be possible to generalize the Lemma to include the union of the two environments, thus making it stronger. In a somewhat similar way, it is possible to generalize about the movements of a single piece, if more than one Lemma exists with respect to its moves. Lemmas are also posited about winning moves. Whenever a move is suggested at some future point in the search, the lemma file is first examined to determine if a lemma exists about this move and if the partial board description matches. If so, the result of the move is assumed known and the search can be foregone. A similar system has been described by Georgy Adelson-Velsky, Vladimir Arlazarov and Mikhail Donskoy [7] [8].

Quotes

Hans Berliner on CAPS-II in his Oral History, March 2005 [9]:

```
And here we are in 1970 and the Northwestern people have just made a big splash and everybody realized that they we
```

```
There was one thing in this dissertation that I'm proud of and that was something that I called 'the causality faci
```

```
So I started thinking about that and when I did my dissertation I had descriptions – as I said I had a lot of struc
```

See also

- Paradise

Publications

- Hans Berliner (**1973**). *Some Necessary Conditions for a Master Chess Program*. 3. IJCAI 1973 (http://dblp.uni-trier.de/db/conf/ijcai/ijcai73.html)
- Hans Berliner (**1974**). *Chess as Problem Solving: The Development of a Tactics Analyser*. Ph.D. thesis, Carnegie Mellon University
- Hans Berliner (**1977**). *A Representation and Some Mechanisms for a Problem-Solving Chess Program*. Advances in Computer Chess 1
- Hans Berliner, Richard Greenblatt, Jacques Pitrat, Arthur Samuel, David Slate (**1977**). *Panel on Computer Game Playing*. 975-982 5. IJCAI 1977 (http://www.sigmod.org/dblp/db/conf/ijcai/ijcai77.html#BerlinerGPSS77), pdf (http://ijcai.org/Past%20Proceedings/IJCAI-77-VOL2/PDF/087.pdf)
- Editor (**1979**). *Chess Ratings from Problem Solving*. Personal Computing, Vol. 3, No. 12, pp. 77 » Chess Problems, Compositions and Studies

Forum Posts

- Re: Method of Analogies?? (https://www.stmintz.com/ccc/index.php?id=19484) by Steven J. Edwards, CCC, May 29, 1998

- Re: Symbolic: Status Report 2007.04.25 (http://www.talkchess.com/forum/viewtopic.php?topic_view=threads&p=114428&t=13382) by Steven Edwards, CCC, April 26, 2007 » Symbolic

External Links

Caps

- Caps (disambiguation) from Wikipedia (https://en.wikipedia.org/wiki/Caps)
- CAPS entreprise - many-core programming (http://www.caps-entreprise.com/)
- Caps lock from Wikipedia (https://en.wikipedia.org/wiki/Caps_lock)
- Calcyphosin (CAPS gene) from Wikipedia (https://en.wikipedia.org/wiki/Calcyphosin)
- Computer Animation Production System - Wikipedia (https://en.wikipedia.org/wiki/Computer_Animation_Production_System)
- Java Caps from Wikipedia (https://en.wikipedia.org/wiki/Java_Caps) » Java
- Problem solving from Wikipedia (https://en.wikipedia.org/wiki/Problem_solving)

Cap

- Cap (disambiguation) from Wikipedia (https://en.wikipedia.org/wiki/Cap_%28disambiguation%29)
- Cap from Wikipedia (https://en.wikipedia.org/wiki/Cap)

References

1. An example of a spherical cap by Pbroks13 (https://commons.wikimedia.org/wiki/User:Pbroks13), October 2008, Spherical cap from Wikipedia (https://en.wikipedia.org/wiki/Spherical_cap)
2. Hans Berliner (**1974**). *Chess as Problem Solving: The Development of a Tactics Analyser*. Ph.D. thesis, Carnegie Mellon University
3. Hans Berliner (**1977**). *A Representation and Some Mechanisms for a Problem-Solving Chess Program*. Advances in Computer Chess 1
4. Hans Berliner, Richard Greenblatt, Jacques Pitrat, Arthur Samuel, David Slate (**1977**). *Panel on Computer Game Playing*. 975-982 5. IJCAI 1977 (http://www.sigmod.org/dblp/db/conf/ijcai/ijcai77.html#BerlinerGPSS77), pdf (http://ijcai.org/Past%20Proceedings/IJCAI-77-VOL2/PDF/087.pdf)
5. Hans Berliner (**1977**). *A Representation and Some Mechanisms for a Problem-Solving Chess Program*. Advances in Computer Chess 1
6. Hans Berliner (**1977**). *A Representation and Some Mechanisms for a Problem-Solving Chess Program*. Advances in Computer Chess 1
7. Georgy Adelson-Velsky, Vladimir Arlazarov, Mikhail Donskoy (**1975**). *Some Methods of Controlling the Tree Search in Chess Programs*. Artificial Intelligence (https://en.wikipedia.org/wiki/Artificial_Intelligence_%28journal%29), Vol. 6, No. 4, pp. 361-371. ISSN 0004-3702. Reprinted (**1988**) in Computer Chess Compendium pp. 129-135
8. Method of Analogies?? (https://www.stmintz.com/ccc/index.php?id=19469) by Bruce Cleaver, CCC, May 29, 1998
9. Oral History of Hans Berliner (http://www.computerhistory.org/chess/related_materials/oral-history/hans_berliner.oral_history.2005.102630824/index.php?iid=orl-43343bb768f00), Interviewed by: Gardner Hendrie (http://www.computerhistory.org/trustee/gardner-hendrie), Recorded: March 7, 2005, The Computer History Museum, pdf (http://archive.computerhistory.org/projects/chess/related_materials/oral-history/hans_berliner.oral_history.2005.102630824/berliner.oral_history_transcript.2005.103630824.pdf), pp. 25.

Up one Level

Retrieved from "https://www.chessprogramming.org/index.php?title=CAPS&oldid=15894"

- This page was last edited on 7 December 2019, at 14:21.
- Content is available under Creative Commons Attribution-ShareAlike 3.0 Unported (CC BY-SA 3.0) unless otherwise noted.